

Universidade de São Paulo
ICMC - Instituto de Ciências Matemáticas e Computação

SCC0240 - Bases de Dados

Prof. Dra. Elaine Parros M. de Sousa
PAE: Anderson Henrique Giacomini

SOS Abobrinha

UMA PLATAFORMA DE REDISTRIBUIÇÃO DE EXCEDENTES AGRÍCOLAS

Beatriz Alves dos Santos	15588630
Daniel Jorge Manzano	15446861
Jhonatan Barboza da Silva	15645049
Kevin Ryoji Nakashima	15675936
Newton Eduardo Pena Villegas	15481732

São Carlos, 11 de Março de 2026

Sumário

1	Introdução	1
2	Modelo Entidade-Relacionamento	1
2.1	Levantamento de requisitos	1
2.2	Principais funcionalidades	3
2.3	Possíveis inconsistências e notas do MER	4
2.4	Diagrama do Modelo Entidade-Relacionamento	5
3	Mudanças Relacionadas à Primeira Entrega	7
3.1	Adaptações de Entidades	7
3.2	Adaptações de IDs Sintéticos	7
3.3	Correção de Ciclo	7
3.4	Adaptações de Participação	7
3.5	Adaptações de Atributos	8
3.6	Adaptações de Relacionamentos	8
3.7	Correções nas Notas	9
3.8	Correção no Levantamento de Requisitos	10
3.9	Adição de Especialização de Funcionário	10
3.10	Relacionamento entre pagamentos e Funcionário de Finanças	10
4	Modelo Relacional	11
4.1	Esquema Relacional	11
4.2	Justificativas Sobre os Mapeamentos	12
5	Mudanças Relacionadas à Segunda Entrega	22
6	Implementação	22
6.1	Ambiente de Desenvolvimento e Tecnologias	23
6.1.1	Front-End	23
6.1.2	Back-End	23
6.1.3	Demais Tecnologias	23
6.1.4	Requisitos para execução	23
6.2	Organização de diretórios	23
6.3	Consultas	24
6.3.1	Consulta de quantidade adquirida e cadastrada por produto	24
6.3.2	Consulta de beneficiários de um lote específico	25
6.3.3	Consulta de beneficiários que requisitaram todos os lotes de uma classificação (Divisão Relacional)	26
6.3.4	Consulta de lotes com preço por unidade acima da média do produtor	27

6.3.5	Consulta de beneficiários com requisição acima da média	28
6.4	Aplicação	29
6.4.1	Funcionalidade de inserção de lotes	30
6.4.2	Funcionalidade de visualização de produtos	32
6.5	Tratamento de Vulnerabilidades	33
6.6	Controle Transacional	34
7	Conclusão	35
8	Referências	35

1 Introdução

No contexto global, o Brasil ocupa uma posição de destaque no setor agrícola, sendo considerado o quarto maior produtor agrícola do mundo. O país se sobressai na produção de diversas commodities, como soja, milho e café, que possuem grande relevância no mercado internacional. No entanto, independentemente do modelo de produção adotado — seja ele baseado em práticas orgânicas ou no uso de defensivos agrícolas — a atividade agrícola gera impactos ambientais significativos.

Além disso, um dos principais desafios relacionados à produção agrícola brasileira é o elevado desperdício de alimentos. De acordo com dados do Instituto Brasileiro de Geografia e Estatística (IBGE), aproximadamente 30% de toda a produção agrícola do país é desperdiçada. Esse cenário evidencia um problema relevante, pois grande parte dos recursos naturais utilizados na produção, como água, solo e energia, acaba sendo utilizada sem gerar o benefício esperado.

Diante desse contexto, este projeto propõe o desenvolvimento de um sistema que servirá como base para uma empresa fictícia voltada à redistribuição sustentável de excedentes agrícolas. O sistema atua como uma plataforma de gestão e intermediação que, embora centralizado na empresa, oferece interfaces específicas com acessos restritos para que produtores rurais e beneficiários interajam diretamente com a plataforma. A proposta consiste em intermediar a relação entre quem produz e quem necessita, direcionando produtos que seriam descartados para locais onde ainda possuem utilidade e valor social.

Nesse sentido, o sistema preza pelo auxílio a instituições sociais (como escolas, hospitais, abrigos, órgãos governamentais de amparo etc.) como forma de garantir que recursos que seriam desperdiçados cumpram uma função social direta e cheguem a quem mais precisa. Além disso, as demais categorias de excedentes - como os destinados ao consumo animal ou à compostagem - também são aproveitadas nesta proposta, sendo redistribuídas para pequenos produtores em situação de vulnerabilidade, de modo a oferecer maior acessibilidade a insumos importantes em suas respectivas produções.

Nesse modelo, as transações não têm finalidade lucrativa, sendo os valores envolvidos utilizados apenas para cobrir custos logísticos e operacionais da redistribuição. Dessa forma, busca-se reduzir o desperdício de alimentos, melhorar a gestão de recursos e contribuir para práticas mais sustentáveis no setor agrícola. Assim, a empresa atribui, àquilo que seria considerado lixo, **valor real**.

2 Modelo Entidade-Relacionamento

2.1 Levantamento de requisitos

Os **produtores rurais** devem registrar seus **produtos** no sistema, de forma que a relação entre produtor e produto caracterize o **lote de produto**, que representa o exce-

dente disponível para redistribuição. Um **produto** por si só não é o objeto da transação, mas sim o **lote**, que consiste em uma quantidade específica de um item vinculado a um determinado **produtor**. Após esse registro, um **funcionário** responsável pela **triagem** deverá realizar a classificação técnica do lote, determinando se o lote é adequado para **consumo humano**, **consumo animal** ou **compostagem**.

Lotes de produto classificados para **consumo humano** deverão ser transportados até um **centro de beneficiamento e distribuição**, onde poderão ser preparados e armazenados temporariamente. Após o processamento, o lote poderá ser marcado no sistema como disponível para solicitação por **instituições sociais** cadastradas. Lotes de produto classificados para **consumo animal** deverão ser transportados até um **armazém**, onde poderão ser preparados e armazenados temporariamente. Após o armazenamento, o lote poderá ser marcado como disponível para **pequenos pecuaristas** cadastrados. Lotes de produto classificados para **compostagem** deverão ser transportados até o centro de compostagem, onde haverá o processo de compostagem e o produto resultante (adubo) poderá ser armazenado temporariamente. Após o armazenamento, o lote poderá ser marcado como disponível para **pequenos agricultores** cadastrados.

Cada **produto** possui nome e tipo (vegetais, frutas, grãos...), já o **lote de produtos** deverá possuir informações como identificação do lote, quantidade, data de colheita, validade, classificação, localização atual e produtor de origem. O **produtor rural** deverá possuir dados como CPF/CNPJ, nome, tipo de produção, contato e endereço. Os **funcionários** da empresa deverão possuir CPF, nome e função, podendo exercer funções relacionadas à **triagem**, **distribuição**, **regularização** ou **finanças**.

Para adquirir **produtos**, o solicitante deve registrar uma **solicitação de aquisição**, que será devidamente verificada e possivelmente autorizada por um **funcionário de regularização**. A solicitação deve informar os **lotes de produtos** a serem adquiridos, suas quantidades (porção do lote) e a declaração de finalidade da aquisição. A solicitação deve ser feita com base nos lotes de produtos disponíveis no sistema, podendo, uma só solicitação, requisitar porções (ou o conteúdo completo) de diferentes lotes, contanto que estes contenham a mesma classificação. Por exemplo: dados dois lotes de produtos, um de batata e outro de cenoura, é possível que uma solicitação requirite metade de cada lote, ou ambos inteiros; mas essa mesma solicitação não poderia requisitar um lote de adubo junto.

A aceitação, pelo **funcionário de regularização**, de uma solicitação de aquisição cria um **lote de entrega**, que consiste na junção de todos os produtos requisitados na mesma solicitação. O lote de entrega contém informações referentes a todos os **lotes de produtos** representados em seu conteúdo, incluindo a quantidade de cada lote de produto presente em si (porção do lote) e deverá possuir o tamanho total e status da entrega. Por exemplo: numa solicitação de metade de um lote de batata e metade de

um lote de cenoura, o lote de entrega resultante contém as informações originais desses lotes e a informação de que ele contém metade da quantidade do lote de batata original e metade da quantidade do lote de cenoura original.

As **transportadoras** deverão ser identificadas por CNPJ e possuir informações como contato, tipo de veículo e endereço. O sistema também deverá registrar as operações de transporte (que poderá ser classificado como transporte de recebimento ou transporte de entrega), incluindo informações como origem, destino, valor de frete, datas de coleta e entrega e o identificador (placa) do veículo responsável. Serviços de transporte poderão ser solicitados por um **funcionário de distribuição**.

Além da gestão logística, o sistema deverá realizar o controle financeiro das operações. Um **funcionário** do setor de **finanças** será responsável por registrar, por meio do gerenciamento das **contas bancárias** da empresa, o ressarcimento dos custos de produção aos produtores rurais e o recebimento dos pagamentos realizados pelas entidades beneficiadas, correspondentes apenas aos custos operacionais da redistribuição. A conta bancária deverá possuir os titulares, tipo, código do banco, número da conta e o número da agência. O sistema também deverá permitir o recebimento de doações financeiras realizadas por **filantropos**.

O sistema também deverá armazenar informações sobre seus **centros logísticos** (que podem ser diferenciados pelo tipo de produto que armazenam: **centros de beneficiamento e distribuição, armazéns** ou **centros de compostagem**), incluindo endereço, contato, capacidade e ocupação. As entidades responsáveis por receber os produtos serão registradas no sistema como beneficiários. Cada beneficiário deverá possuir nome, classificação, endereço, contato, uma validação que comprove sua elegibilidade para participar do sistema e um documento identificador (CPF/CNPJ).

2.2 Principais funcionalidades

- Produtor Rural:
 - Cadastro e remoção de produtos no sistema (ação regularizada pelo funcionário de regularização).
 - Cadastro de lotes de produtos, considerando os produtos já existentes no sistema.
 - Pesquisa e remoção de lotes de produtos próprios.
- Beneficiário:
 - Pesquisa sobre os lotes de produtos disponíveis.
 - Registro de uma solicitação de aquisição sobre um ou mais lotes disponíveis.
- Funcionário de Regularização:
 - Verificar (aceitar ou recusar) solicitações de aquisição feitas pelos destinatários.
 - Regularizar os cadastros de produtos feitos pelo produtor rural no sistema.

- Regularizar os acessos de produtores e beneficiários na plataforma.
- Funcionário de Finanças:
 - Gerencia a conta bancária da empresa, que, por sua vez, realiza e recebe transações de outras entidades do sistema.
- Funcionário de Triagem:
 - Validar e classificar os lotes de produtos registrados pelos produtores.
- Funcionário de Distribuição:
 - Solicitar, a uma transportadora, um transporte e atribuí-lo a um lote de produto ou de entrega.
- Funcionário de Administração:
 - Para cada tipo de produto, consultar a quantidade adquirida e distribuída pela empresa em certo mês.
 - Dado um lote de produto cadastrado, consultar quais beneficiários (solicitações de aquisição aprovadas) o adquiriram.

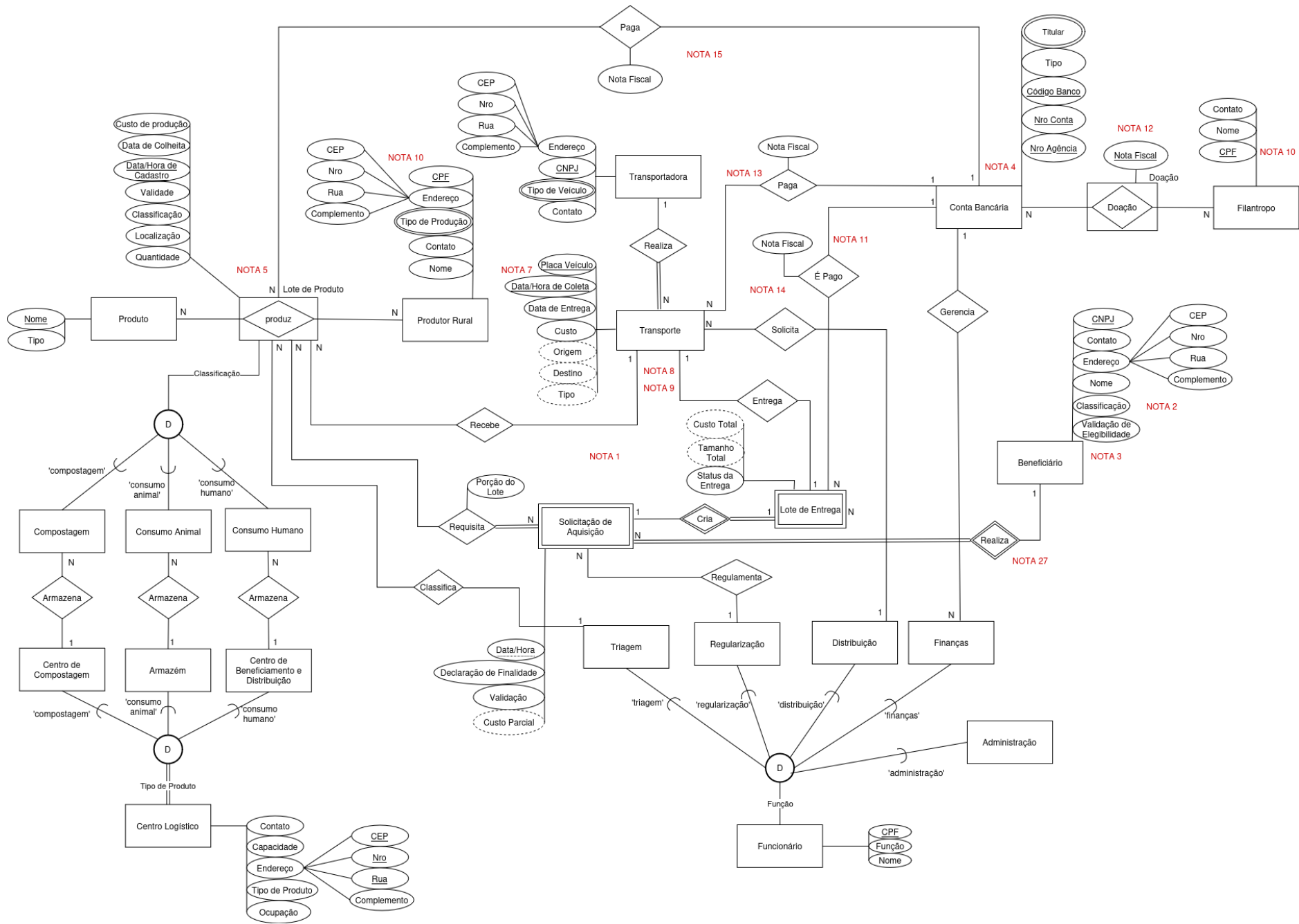
2.3 Possíveis inconsistências e notas do MER

- **Ciclo Lote de Produto - Solicitação de Aquisição - Lote de Entrega - Transporte (Nota 1):** Consideramos que um dado transporte é feito especificamente para entrega de Lotes de Produto ou Lotes de Entrega, sem sobreposição. Dessa forma, este ciclo não é problemático para o sistema, uma vez que ele não deve existir na realidade.
- **Nota 2:** A entidade Beneficiário deve ser classificada como pequeno agricultor, pequeno pecuarista ou instituição social.
- **Nota 3:** A entidade Beneficiário só pode solicitar e ter acesso a produtos correspondentes a sua classificação.
- **Nota 4:** A entidade Conta Bancária, representando as finanças da empresa, paga pelos Lotes de Produto e recebe pelos Lotes de Entrega.
- **Nota 5:** A chave de Lote de Produto é a combinação entre os campos Nome (Produto), CPF/CNPJ (Produtor) e Data/Hora de Cadastro.
- **Nota 6:** Apagada (em relação à última entrega). A identificação alternativa proposta na antiga Nota 6 foi adaptada para se tornar a chave primária de Lote de Produto.
- **Nota 7:** É papel da aplicação garantir que um transporte não possa ser registrado com um dado veículo se ainda houver um transporte pendente (ou seja, que ainda não chegou à data de entrega) que esteja associado a este veículo.
- **Nota 8:** O transporte pode "receber" lotes de produto para a empresa (buscando do produtor).

- **Nota 9:** O transporte pode "entregar" lote de entrega para o beneficiário (buscando do centro da empresa).
- **Nota 10:** É papel do sistema gerenciar qual o tipo de documento existente para cada entidade com campo nomeado "CPF/CNPJ".
- **Nota 11:** Um Lote de Entrega é pago pelo Beneficiário que está relacionado à ele por meio da Solicitação de Aquisição.
- **Nota 12:** A chave de Doação é Nota Fiscal.
- **Nota 13:** O pagamento do transporte pode ser relativo tanto à entrega ao centro logístico (buscar no produtor) quanto à entrega ao beneficiário.
- **Nota 14:** O ciclo Transporte - Lote de Entrega - Conta Bancária não representa um problema, pois os relacionamentos guardam informações diferentes e as contas bancárias relacionadas não são necessariamente as mesmas. Entretanto, há algumas restrições que devem ser garantidas pela aplicação: qualquer pagamento só pode ser realizado após o estabelecimento do relacionamento Entrega, pois é ele que define o custo da entrega. Além disso, o valor do pagamento do transporte, que deve corresponder ao custo (atributo de Transporte), não pode exceder o valor do pagamento do lote, já que este o inclui.
- **Nota 15:** O ciclo Transporte - Lote de Produto - Conta Bancária não representa um problema, pois os relacionamentos guardam informações diferentes e as contas bancárias relacionadas não são necessariamente as mesmas. Entretanto, há algumas restrições a serem observadas: o pagamento pelo transporte só pode ser feito após o relacionamento Recebe ser estabelecido e deve ser equivalente ao custo do transporte.
- **Nota 16:** Os demais ciclos presentes no diagrama não foram mencionados por representarem ciclos naturais da modelagem, que armazenam informações distintas e não necessariamente relacionadas entre si. Dessa forma, eles não caracterizam ciclos de redundância nem de dependência.

2.4 Diagrama do Modelo Entidade-Relacionamento

A imagem do Modelo Entidade-Relacionamento do sistema. [Link para melhor visualização do diagrama:](#) Mer



3 Mudanças Relacionadas à Primeira Entrega

Após a primeira entrega, usando como base as correções apontadas pelo monitor da disciplina e novas ideias de ajustes propostas pelo grupo, foram realizadas diversas mudanças para o prosseguimento do projeto.

3.1 Adaptações de Entidades

- **Solicitação de Aquisição:** foi transformada em uma entidade fraca de **Beneficiário** para evitar o uso de ID sem sentido semântico.
- **Transporte:** foi transformada em uma entidade forte, porque a sua nova chave composta (**Data/Hora de Coleta, Placa Veículo**) a identifica totalmente, sem necessidade da entidade Transportadora.

3.2 Adaptações de IDs Sintéticos

Foi apontado, pelo monitor, a recorrência de IDs artificiais em várias das entidades do MER. Para corrigir isto, tomamos as seguintes alternativas para identificação:

- **ID Lote:** tornou-se a combinação entre a chave de Produtor Rural (**CPF/CNPJ**), a chave parcial **Data/Hora de Cadastro** e a chave de Produto (**Nome**).
- **ID Veículo:** tornou-se **Placa Veículo**.
- **ID Solicitação:** tornou-se a combinação entre a chave de Beneficiário (**CPF/CNPJ**) e a chave parcial **Data/Hora**. Assim, a Solicitação é identificada pelo Beneficiário que a criou e o horário que foi feita.

3.3 Correção de Ciclo

- Foi retirado o relacionamento “Compõe” entre **Lote de Produto** e **Lote de Entrega**, desfazendo o ciclo existente entre Lote de Produto, Solicitação de Entrega e Lote de Entrega. Uma vez que é possível, para o Lote de Entrega, obter as informações dos Lotes de Produto a partir da navegação lógica por Solicitação de Aquisição, concluímos que manter o relacionamento causaria um ciclo desnecessário.
 - A partir disso, a Nota 1, que apontava a ocorrência deste ciclo solucionado, foi adaptada para explicar outro ciclo.

3.4 Adaptações de Participação

- **Solicitação de Aquisição - Lote de Produto:** foi adicionada a participação total por parte da Solicitação uma vez que ela só pode ocorrer com base em, no mínimo, um Lote de Produto.
- **Especialização de Lote de Produto:** foi retirada a especialização total do Lote de Produto uma vez que ele não possui uma especialização enquanto não for classificado

pelo funcionário de triagem (ou seja, desde que ele foi cadastrado até o momento que é triado, ele não possui uma especialização, sendo inconsistente considerá-la como total).

3.5 Adaptações de Atributos

- **Frete (Transporte):** foi renomeado para **Custo** para maior clareza, como sugerido pelo monitor.
- **Data de Entrega (Transporte):** deixou de ser parte da chave, como sugerido pelo monitor. Agora, o sistema permite que um transporte possa ser registrado no sistema sem uma data de entrega (que poderá ser, então, adicionada posteriormente na entrega).
- **Data de Coleta (Transporte):** passou a ser **Data/Hora de Coleta**. Dessa forma, fica possível que um dado veículo possa realizar mais de um transporte num mesmo dia (contanto que os horários de coletas sejam diferentes).
- **Tipo (Transporte):** foi adicionado para indicar se um transporte é de recebimento ou de entrega. Como é uma informação inferida pela navegação lógica com as entidades que entregam ou recebem, é considerado um atributo derivado.
- **Origem e Destino (Transporte):** foram transformados em atributos derivados pela possibilidade de obtenção destas informações por meio da navegação lógica pelas entidades de Origem (Produtor Rural, Centro Logístico) e Destino (Centro Logístico, Beneficiário).
- **Tipo de Transporte (Transportadora):** tornou-se **Tipo de Veículo**. Alteração feita unicamente para melhor entendimento do campo.
- **Nota Fiscal (Doação):** passou a ser chave da nova agregação Doação.
- **Custo Parcial (Solicitação de Aquisição):** foi adicionado para registrar o custo parcial a ser pago pelo beneficiário. É derivado dos custos associados aos produtos escolhidos (custo de produção e de transporte ao centro logístico). É parcial, pois o valor total da aquisição vai depender do custo de entrega, calculado posteriormente.
- **Custo Total (Lote de Entrega):** foi adicionado para registrar o custo total a ser pago pelo beneficiário. É derivado do custo parcial da Solicitação de Aquisição associada e do custo de entrega do Transporte associado.
- **Capacidade de Carga (Transportadora):** foi removido por não trazer sentido semântico agregador.

3.6 Adaptações de Relacionamentos

- **Filantropo - Conta Bancária:** O nome do relacionamento foi alterado para **Doa** e a cardinalidade anterior (N-1) foi adaptada para N-N para ser compatível ao fato de a

empresa poder possuir mais de uma Conta Bancária associada e, conseqüentemente, ser possível que o Filantropo realize doações para as N Contas Bancárias. Além disso, foi transformado em uma agregação **Doação**, com uma chave própria, para permitir múltiplas doações, de um mesmo Filantropo, para uma mesma Conta Bancária.

- **Beneficiário - Conta Bancária:** este relacionamento, que representa o pagamento de um Lote de Entrega para a empresa, foi apagado e substituído pelo relacionamento **É Pago** entre **Lote de Entrega - Conta Bancária**. Este novo relacionamento apresenta de forma mais clara o pagamento de um dado Lote de Entrega específico para a Conta Bancária e soluciona o problema de que um Beneficiário podia realizar apenas um pagamento para uma dada Conta, como apontado pelo monitor na primeira entrega. Além disso, o autor do pagamento (Beneficiário) ainda pode ser identificado por navegação lógica.
- **Filantropo - Conta Bancária:** foi transformado numa agregação **Doação**, com uma chave própria, para permitir que um mesmo Filantropo possa realizar mais de uma doação para a mesma Conta Bancária.
- **Transporte - Lote de Entrega:** A cardinalidade anterior (1-N) foi adaptada para 1-1, de forma que um Transporte de entrega corresponda a apenas um destino (Lote de Entrega) por vez.
- **Conta Bancária - Transportadora:** Este relacionamento foi substituído pelo relacionamento **Paga**, entre Conta Bancária - Transporte, de forma que representa melhor a operação de pagamento por um Transporte, além de, agora, permitir que a Conta efetue diferentes pagamentos a uma Transportadora (com base nos Transportes efetuados), o que não era possível anteriormente.

3.7 Correções nas Notas

Em decorrência das mudanças feitas para a segunda entrega, algumas notas foram adaptadas e, outras, adicionadas.

As notas já corrigidas se encontram na subseção 2.3. A seguinte listagem apenas aponta as mudanças feitas brevemente.

- **Nota 1:** Explicava um ciclo anterior que foi corrigido. Foi reformulada para abordar outro ciclo.
- **Nota 4:** Adaptada para abordar os relacionamentos atualizados da entidade Conta Bancária.
- **Nota 5:** Apresentava a identificação de Lote de Produto. Foi atualizada para representar a nova escolha de chave.
- **Nota 6:** Apagada. Como apontado anteriormente na própria nota corrigida, a identificação alternativa proposta anteriormente foi adaptada para se tornar a chave primária de Lote de Produto.

- **Nota 7:** Substituída por outra nota relacionada ao que havia sido proposto.
- **Nota 8:** Apenas adiciona o plural de "lote", como apontado pelo monitor.
- **Notas 11, 12, 13, 14, 15 e 16:** Novas notas adicionadas com as novas adições da segunda entrega.

3.8 Correção no Levantamento de Requisitos

Assim como com as notas, é importante apontar que o levantamento de requisitos (subseção 2.1) também foi adaptado em alguns pontos, seja para atender a observações do monitor, seja para adaptar as descrições às mudanças desta entrega.

3.9 Adição de Especialização de Funcionário

Foi adicionada ao diagrama uma nova especialização de Funcionário: o **Funcionário de Administração**. Embora ele não participe de nenhum relacionamento e, portanto, não seja mapeado no modelo relacional, sua presença foi incluída no diagrama para explicitar sua existência, já que foram adicionadas ao sistema funcionalidades atribuídas a esse tipo de funcionário. Sua inclusão foi feita para adicionar operações de busca mais complexas ao sistema, conforme sugerido pelo monitor, para que sejam implementadas em uma entrega futura.

3.10 Relacionamento entre pagamentos e Funcionário de Finanças

Embora reconheçamos que registrar explicitamente o Funcionário de Finanças responsável por cada pagamento agregaria rastreabilidade ao sistema, optamos por manter os relacionamentos de pagamento ("Paga" e "É Pago") diretamente com a Conta Bancária pelas seguintes razões:

- Ligar os pagamentos ao Funcionário de Finanças em vez da Conta Bancária introduziria o problema de que, caso um funcionário gerencie mais de uma conta - o que não é impedido estruturalmente pelo modelo -, a informação de qual conta específica foi utilizada no pagamento seria perdida, exigindo verificação adicional pela aplicação. Na modelagem atual, essa informação está registrada de forma direta e inequívoca.
- A informação de qual funcionário está associado a um pagamento ainda é recuperável por navegação lógica, passando de Conta Bancária para o Funcionário que a gerencia, sem necessidade de alteração estrutural no modelo.

4 Modelo Relacional

4.1 Esquema Relacional

A imagem do Modelo Relacional. Link para melhor visualização do mapeamento: [link aqui](#).

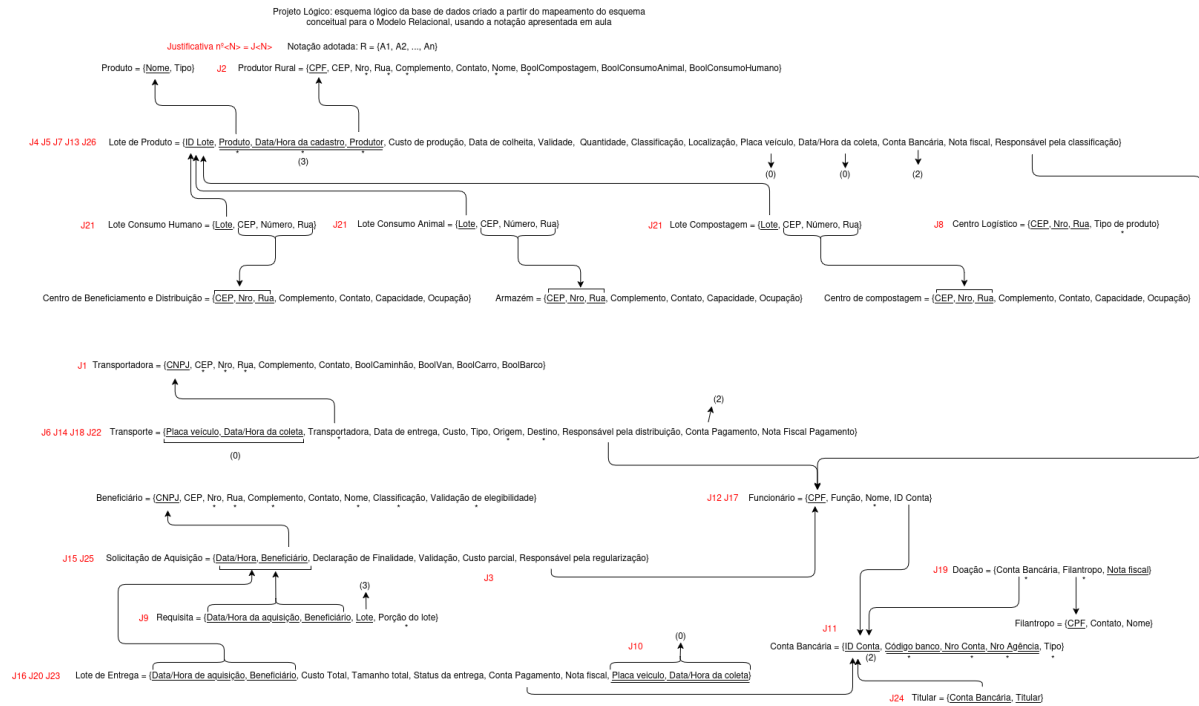


Figure 2: Diagrama de Mapeamento

4.2 Justificativas Sobre os Mapeamentos

- **Justificativa 1 - Atributo multivalorado “Tipo de Veículo” de Transportadora:** O mapeamento do atributo multivalorado “Tipo de Veículo” foi feito dentro do esquema Transportadora com atributos booleanos, indicando os possíveis tipos de veículos que a entidade do conjunto Transportadora oferece (caminhão, van, carro, barco).
 - **Alternativas:** Criar um novo esquema chamado “Tipo de Veículo” cuja chave primária seria composta pela chave estrangeira CNPJ (referência de Transportadora) e pelo atributo Tipo (que definiria os tipos veículos existentes). Essa opção não foi adotada por criar a necessidade de JOINS quando realizadas buscas.
 - **Vantagens:** Como essa busca é necessária para que a aplicação escolha a transportadora (e, conseqüentemente, é frequente), a alternativa escolhida é vantajosa por poupar JOINS e acelerar a busca. Além disso, o uso de atributos booleanos para representar os tipos faz com que haja uma maior economia no quesito armazenamento.
 - **Desvantagens:** Há a ocorrência de valores nulos quando uma transportadora não oferece algum dos tipos de veículo.
- **Justificativa 2 - Atributo multivalorado “Tipo de Produção” de Produtor Rural:** O mapeamento do atributo multivalorado foi feito a partir da criação de atributos booleanos, no esquema de Produtor Rural, indicando os tipos de produtos que o produtor produz (Compostagem, Consumo Animal e Consumo Humano).
 - **Alternativas:** Criar um novo esquema chamado Tipo de Produção, cuja chave primária é composta pela união entre a chave de Produtor e o atributo Tipo.
 - **Vantagens:** Durante as buscas, torna-se não necessário a realização de JOINS, gerando mais rapidez nas operações. Além disso, com a criação de atributos booleanos, não há a necessidade de criar uma nova tabela para o atributo multivalorado, economizando espaço de armazenamento.
 - **Desvantagens:** Há a ocorrência de valores nulos quando um produtor não produz algum dos tipos de produção.
- **Justificativa 3 - Atributo “Custo Parcial” de Solicitação de Aquisição:** O atributo derivado foi mapeado de modo a manter o valor do atributo armazenado no esquema de Solicitação de Aquisição.
 - **Alternativas:** Não armazenar o valor do atributo no banco de dados e, sempre que houver a necessidade de uso do atributo, realizar o cálculo em aplicação.
 - **Vantagens:** Otimização do acesso à informação (não exige o cálculo para acessá-la).
 - **Desvantagens:** Caso o valor mude, deve ser recalculado e atualizado. Porém,

como os valores dos Lotes de Produto não tendem a mudar após seu cadastro, o custo deve permanecer constante e, portanto, essa desvantagem se torna mínima.

- **Justificativa 4 - Mapeamento da agregação Lote de Produto entre Produto e Produtor Rural:** O mapeamento da agregação foi feito criando um novo esquema Lote de Produto, mantendo, como chave primária, o identificador artificial ID Lote e, como chave secundária, a junção dos atributos Produto, FK do Produto, Data/Hora de Cadastro e Produtor, FK do Produtor.

- **Alternativas:** Mapear o Lote de Produto em um esquema e o relacionamento Produz em um esquema à parte, o que aumentaria a quantidade de JOINS para extrair informações do Produtor ou do Produto a partir de um Lote de Produto.
- **Vantagens:** A quantidade de JOINS para buscas que envolvam o Produtor ou Produto relacionado a um Lote de Produtos diminui, além de poupar a criação de um outro esquema (o esquema “Produz”).
- **Desvantagens:** As buscas que envolvam Produtos de um Produtor ficam mais longas, porque a tabela Lote de Produto tem mais registros do que a tabela Produz teria. Entretanto, como essa busca não é tão frequente, a alternativa adotada (omitir “Produz”) é a melhor.

- **Justificativa 5 - Mapeamento do relacionamento “Classifica” entre Funcionário de Triagem e Lote de Produto - 1:N:** O mapeamento do relacionamento “Classifica” foi feito a partir da inserção da chave primária de Funcionário como chave estrangeira (Responsável pela Classificação) no esquema Lote de Produto, procedimento padrão de mapeamento de um relacionamento 1:N.

- **Alternativas:** Criar um esquema “Classifica”.
- **Vantagens:** Diminui a quantidade de JOINS nas buscas de Lotes triados por um Funcionário de Triagem. Além disso, torna-se não necessário a criação de um novo esquema, gerando economia de espaço de armazenamento.
- **Desvantagens:** Se a quantidade de lotes triados for pouca, o desperdício de memória é alto e a busca pode ficar mais demorada. Nesse caso, entretanto, os Lotes de Produtos devem ser triados pouco tempo após serem registrados no sistema e, por isso, existem poucos lotes não triados, sendo a melhor alternativa a adotada.

- **Justificativa 6 - Mapeamento do relacionamento “Realiza” entre Transportadora e Transporte - 1:N:** O mapeamento do relacionamento “Realiza” foi feito a partir da inserção da chave primária de Transportadora, como chave estrangeira (CNPJ), no esquema de Transporte, mantendo o atributo como não nulo para garantir a participação total de Transporte no relacionamento.

- **Alternativas:** Não há alternativa que garanta participação total no próprio

mapeamento, apenas em aplicação. Assim, não consideramos outra alternativa a comparar.

- **Vantagens:** Garante participação total.

- **Desvantagens:** Não há desvantagens.

- **Justificativa 7 - Mapeamento do relacionamento “Recebe” entre Transporte e Lote de Produto - 1:N:** O Mapeamento do relacionamento “Recebe” foi feito a partir da inserção da chave primária de Transporte como chave estrangeira composta (Data/Hora da coleta + Placa Veículo) no esquema de Lote de Produto, procedimento padrão de mapeamento de um relacionamento 1:N.

- **Alternativas:** Criar um esquema Entrega.

- **Vantagens:** Diminui a quantidade de JOINS nas buscas de Lotes entregues a um Centro Logístico por um Transporte específico. Além disso, economiza-se espaço ao não criar o esquema “Entrega”.

- **Desvantagens:** Se a quantidade de Lotes de Produto entregues for pouca, o desperdício de memória é alto. Porém, como é esperado que os Lotes de Produto sejam, em maioria, adquiridos, a quantidade de valores nulos são reduzidos.

- **Justificativa 8 - Mapeamento da especialização “Centro Logístico”:** O mapeamento da generalização Centro Logístico foi feito criando esquemas separados para cada entidade específica e um esquema auxiliar Centro Logístico, utilizado para armazenar o atributo de critério “Tipo de produto” e garantir a especialização total.

- **Alternativas:** A outra alternativa mais coerente para esse caso seria mapear apenas a entidade genérica em uma tabela, com o atributo de critério definindo o critério da especialização. Embora essa abordagem implique em criar menos tabelas, também cria a necessidade de verificação em aplicação da consistência do atributo de critério para cada relacionamento com a especialização de Lote de Produto. Além disso, também seria necessário garantir, via aplicação, que o atributo de critério não admita valores que causem sobreposição entre as especializações. Portanto, a quantidade de verificação dependente da aplicação seria mais frequente do que na abordagem escolhida.

- **Vantagens:** A abordagem escolhida já garante que os relacionamentos das especializações sejam feitos com a classificação correta, sem a necessidade de verificar em aplicação. Além disso, ela também garante a participação total e disjunção da especialização, considerando a devida manutenção da tabela auxiliar. Consultas em um tipo específico de Centro Logístico também são beneficiadas nesse modelo, pois eles já estão separados em tabelas próprias.

- **Desvantagens:** Para garantir a disjunção e participação total, é necessário fazer a devida manutenção da tabela auxiliar. Ou seja, os registros existentes

nas tabelas específicas têm, necessariamente, que existir na tabela auxiliar e com o atributo de critério correto. Além disso, a abordagem escolhida implica em criar três tabelas com esquemas iguais. Logo, qualquer mudança na estrutura de um atributo genérico (ex.: mudar o tamanho do campo CEP) precisará ser replicada em todas as três tabelas específicas.

- **Justificativa 9 - Mapeamento do relacionamento “Requisita” entre Lote de Produtos e Solicitação de Aquisição - N:N:** O mapeamento foi realizado através da criação de um novo esquema independente. A chave primária deste novo esquema é composta pela chave primária de Lote de Produto e pela chave primária de Solicitação de Aquisição, ambas como chaves estrangeiras, no esquema “Requisita”. O atributo do relacionamento, Porção do Lote, foi adicionado como uma coluna comum nesta nova tabela.
 - **Alternativas:** No modelo relacional não há outras alternativas válidas de mapeamento para relacionamentos N:N.
 - **Vantagens:** Dado que não existem alternativas, não há como apontar vantagens sobre alguma comparação.
 - **Desvantagens:** A participação total neste relacionamento, assim como em todos relacionamentos N:N com alguma participação total, precisa ser garantida pela aplicação.
- **Justificativa 10 - Mapeamento do relacionamento “Entrega” entre Transporte e Lote de Entrega - 1:1:** O mapeamento foi feito inserindo a chave primária composta de Transporte, como chave estrangeira (Placa Veículo + Data/Hora de Coleta), no esquema de Lote de Entrega.
 - **Alternativas:** Realizar o mapeamento inverso (migrar a chave do Lote para Transporte) ou criar uma tabela associativa separada, que seria vantajosa para casos em que o relacionamento não é tão recorrente. No mapeamento inverso, entretanto, seriam comuns campos nulos em Transporte, dado que existem muitos dos Transportes carregam Lotes de Produto (e não somente de Entrega). Também, para o mapeamento com a tabela associativa, operações de buscas criariam a necessidade de JOINS e haveria maior consumo de memória com a tabela; além disso, como o relacionamento deve ser comum, este tipo de mapeamento não deve ser vantajoso.
 - **Vantagens:** A quantidade de valores nulos no esquema de Lote de Entrega é menor em relação ao mapeamento inverso. Além disso, em comparação com o mapeamento em um esquema separado, são poupados memória e JOINS de buscas.
 - **Desvantagens:** Devido à participação parcial, a coluna da chave estrangeira conterá valores nulos quando o lote não estiver vinculado a um transporte. Além

disso, é papel da aplicação garantir a cardinalidade 1:1.

- **Justificativa 11 - Mapeamento da chave de “Conta Bancária”:** A chave da entidade foi substituída por um identificador artificial chamado ID Conta, tornando a antiga chave, composta pelos atributos Código Banco, Nro Conta e Nro Agência, um identificador secundário da entidade.
 - **Alternativas:** Manter a chave composta anterior.
 - **Vantagens:** Evita a propagação de uma chave composta nas tabelas dependentes, reduzindo o armazenamento e simplificando a integridade referencial. Como a entidade Conta Bancária é amplamente referenciada, o ID artificial garante um índice menor e mais compacto do que um índice sobre três campos. Isso consome menos memória e acelera significativamente as operações de JOIN.
 - **Desvantagens:** Um atributo a mais em Conta Bancária e perda do valor semântico da chave.
- **Justificativa 12 - Mapeamento da generalização “Funcionário”:** O mapeamento da generalização foi feito criando um único esquema, contendo os atributos gerais, os atributos específicos e o atributo de critério.
 - **Alternativas:** Criar um esquema para cada entidade especializada ou criar cada esquema da especialização e uma entidade pai auxiliar.
 - **Vantagens:** A aplicação não precisa garantir a disjunção e há economia de espaço (uma vez que não são criados mais esquemas).
 - **Desvantagens:** A aplicação precisa garantir que os relacionamentos que envolvam uma especialização de funcionário sejam condizentes com o seu atributo de critério. Além disso, haverá a ocorrência de campos nulos relativos aos atributos específicos, porém, por conta da existência de um único atributo específico, esse fator possui um impacto mínimo.
- **Justificativa 13 - Mapeamento do relacionamento “Paga” entre Conta Bancária e Lote de Produto - 1:N:** A chave sintética de Conta Bancária (ID Conta) foi mapeada como chave estrangeira (Conta Bancária), no esquema de Lote de Produto. O atributo do relacionamento, Nota Fiscal, também migrou para o esquema de Lote de Produto.
 - **Alternativas:** Criar um esquema Paga.
 - **Vantagens:** É o mapeamento padrão e mais eficiente para cardinalidades 1:N. Impede a criação de esquemas extras e agiliza as consultas, eliminando a necessidade da realização de um JOIN extra na hora de verificar de qual conta saiu o pagamento de um Lote de Produto.
 - **Desvantagens:** Serão ocasionados campos nulos referentes às informações de compras (Nota Fiscal e ID Conta). Entretanto, como mencionado em J7, é

esperado que os Lotes de Produto sejam majoritariamente adquiridos (sendo esperados menos campos nulos).

- **Justificativa 14 - Mapeamento do relacionamento “Solicita” entre Funcionário de Distribuição e Transporte - 1:N:** A chave de Funcionário de Distribuição foi mapeada como chave estrangeira (Responsável pela distribuição), no esquema de Transporte.
 - **Alternativas:** Criar um esquema Solicita.
 - **Vantagens:** Pela mesma vantagem da Justificativa 13, elimina a necessidade de realização de um JOIN extra para verificar qual funcionário solicitou uma entidade do conjunto Transporte.
 - **Desvantagens:** É necessário que a aplicação garanta que o funcionário do relacionamento tenha a função de “Distribuição”. Além disso, é possível que existam campos nulos para entidades do conjunto Transporte que ainda não foram solicitados. Entretanto, é esperado que os transportes oferecidos tenham sido, em sua maioria, solicitados por algum funcionário, diminuindo essa quantidade de valores nulos.
- **Justificativa 15 - Mapeamento do relacionamento “Regulamenta” entre Funcionário de Regularização e Solicitação de Aquisição - 1:N:** A chave de Funcionário de Regularização foi mapeada como chave estrangeira (Responsável pela regularização), no esquema de Solicitação de Aquisição.
 - **Alternativas:** Criar um esquema Regulamenta.
 - **Vantagens:** Pela mesma vantagem da Justificativa 13, elimina a necessidade de realização de um JOIN extra para verificar qual funcionário aprovou uma entidade do conjunto Solicitação de Aquisição.
 - **Desvantagens:** É necessário que a aplicação garanta que o funcionário do relacionamento tenha a função de “Regulamentação”. Além disso, é possível que existam campos nulos para entidades do conjunto Solicitação de Aquisição que não foram regulamentadas, porém esse fator se trata de um evento de pouca recorrência no contexto do sistema, uma vez que a empresa deve regulamentar a maioria das solicitações criadas.
- **Justificativa 16 - Mapeamento do relacionamento “Cria” entre Lote de Entrega e Solicitação de Aquisição - 1:1:** O mapeamento do relacionamento foi feito inserindo a chave primária composta de Solicitação de Aquisição como chave primária composta (Data/Hora de aquisição + Beneficiário) no esquema de Lote de Entrega uma vez que Lote de Entrega é entidade fraca de Solicitação de Aquisição.
 - **Alternativas:** Não há outras alternativas de mapeamento.
 - **Vantagens:** O sistema impede a criação de uma entidade do conjunto Lote de

Entrega órfão e sem verificação.

- **Desvantagens:** Como se trata de uma entidade fraca que herda chaves de outra entidade fraca, a entidade acaba por conter muitas chaves.

- **Justificativa 17 - Mapeamento do relacionamento “Gerencia” entre Conta Bancária e Funcionário de Finanças - 1:N:** A chave sintética de Conta Bancária (ID Conta) foi mapeada como chave estrangeira (ID Conta), no esquema Funcionário.

- **Alternativas:** As alternativas seriam propagar a chave composta inteira (três colunas) para o esquema Funcionário ou criar um esquema associativo Gerencia.
- **Vantagens:** Em vez de armazenar uma chave composta para cada funcionário, armazena-se apenas o ID, o que traz vantagem em termos de armazenamento. Além disso, o mapeamento escolhido garante a rastreabilidade exata de quais funcionários do setor financeiro gerenciam e possuem acesso a cada conta.
- **Desvantagens:** Devido ao mapeamento da especialização de Funcionário em um único esquema, todas as entidades do conjunto Funcionário que não possuem, como atributo de critério, o valor “Finanças”, terão um campo nulo em ID Conta. Além disso, é papel da aplicação garantir que apenas os funcionários com essa função contenham um valor válido em ID Conta.

- **Justificativa 18 - Mapeamento do relacionamento “Paga” entre Conta Bancária e Transporte - 1:N:** O mapeamento do relacionamento foi feito inserindo a chave primária de Conta Bancária, como chave estrangeira (Conta Pagamento), no esquema Transporte. Além disso, por se tratar de um relacionamento com atributo, houve também a inserção do atributo Nota fiscal, como Nota Fiscal Pagamento, no esquema Transporte.

- **Alternativas:** Mapear o relacionamento em um esquema à parte. Entretanto, esse tipo de mapeamento é aconselhável quando o relacionamento possui pouca participação, o que não é o caso, já que, idealmente, a maioria das entidades do conjunto Transporte serão pagos em algum momento. Assim, essa alternativa implicaria em criar um esquema a mais sem necessidade.
- **Vantagens:** Garante a cardinalidade correta da relação, não há necessidade de JOIN para acessar as informações de um pagamento relativo a um Transporte e não há necessidade de criar uma nova tabela.
- **Desvantagens:** Adiciona mais atributos em uma tabela já relativamente grande. Poderia gerar desperdício de memória, caso houvesse pouca participação no relacionamento, o que, como citado anteriormente nas alternativas, não é o caso, sendo uma desvantagem de pouco impacto.

- **Justificativa 19 - Mapeamento da agregação “Doação” entre Conta Bancária e Filantropo:** O mapeamento foi feito criando um esquema para a agregação,

mantendo as chaves primárias de Conta Bancária e de Filantropo como chaves estrangeiras não nulas no esquema Doação, garantindo que, em cada doação, haja a participação das duas entidades e que permita mais de uma doação pelo mesmo par de entidades. Além disso, foi omitido o mapeamento do relacionamento Doa (que dá origem à agregação).

- **Alternativas:** Mapear, junto com a agregação, o relacionamento Doa, criando um esquema para o relacionamento e para a agregação.
 - **Vantagens:** A quantidade de JOINs para buscas que envolvam os detalhes da doação vinculada às entidades envolvidas diminui, além de simplificar o modelo ao evitar a criação de um esquema redundante para o relacionamento Doa.
 - **Desvantagens:** Sem o mapeamento de “Doa”, cria a necessidade de JOINs extras em casos de busca entre as entidades Conta Bancária e Filantropo (sem necessidade da Nota Fiscal). Contudo, dado que devem ser maiores as buscas que consideram Nota Fiscal, essa operação não deve ser comum e a desvantagem é menor.
- **Justificativa 20 - Mapeamento do relacionamento “É pago” entre Conta Bancária e Lote de Entrega - 1:N:** O mapeamento do relacionamento foi feito inserindo a chave primária de Conta Bancária como chave estrangeira (Conta Pagamento) no esquema Lote de Entrega. Além disso, do mesmo modo que na Justificativa 18, o atributo Nota Fiscal também foi adicionado como atributo do esquema Lote de Entrega.
 - **Alternativas:** Mapear o relacionamento em um esquema à parte. Entretanto, assim como apontado na Justificativa 18, essa estratégia é mais vantajosa em casos de relacionamentos com pouca participação, o que não é o caso, já que a empresa deve esperar que a maioria das entidades do conjunto Lote de Entrega sejam pagas pelas entidades do conjunto Beneficiário.
 - **Vantagens:** Garante a cardinalidade correta da relação, não há necessidade de JOIN para acessar as informações da Conta relativo a um pagamento e não há necessidade de criar uma nova tabela.
 - **Desvantagens:** Adiciona mais atributos em uma tabela já relativamente grande. Poderia gerar desperdício de memória, caso houvesse pouca participação no relacionamento, o que, como citado anteriormente nas alternativas, não é o caso, sendo uma desvantagem de pouco impacto.
 - **Justificativa 21 - Mapeamento dos relacionamentos “Armazena” entre as especializações de Centro Logístico e as especializações de Lote de Produto - 1:N:** Os relacionamentos Armazena entre as entidades específicas da generalização Lote de Produto e suas correspondentes entidades específicas da generalização de Centro Logístico foram mapeados da mesma forma. Por ser um relacionamento

1:N, foram inseridos os atributos pertencentes à chave composta primária de Centro Logístico, como chave estrangeira composta (CEP + Nro + Rua), nos esquemas das entidades específicas de Lote de Produto.

- **Alternativas:** Mapear o relacionamento em um esquema à parte. Entretanto, assim como os casos das Justificativas 18 e 20, esse relacionamento tem grande participação dado que se espera que Lotes de Produto já classificados (ou seja, especializados) sejam, em maioria, armazenados, assim sendo menos vantajosa a criação de um esquema associativo.
 - **Vantagens:** Garante a cardinalidade correta da relação, não há necessidade de JOIN para acessar as informações do local de armazenamento relativo a um Lote específico e não há necessidade de criar uma nova tabela.
 - **Desvantagens:** Adiciona mais atributos aos esquemas das especializações de Lote; mas, como esses esquemas são pequenos, essa desvantagem é menor. Além disso, é papel da aplicação garantir que as entidades do conjunto Lote de Produto estejam devidamente vinculadas às entidades específicas corretas de Centro Logístico.
- **Justificativa 22 - Atributos derivados Origem, Destino e Tipo de Transporte:** Os atributos derivados foram mapeados de modo a manter armazenados os valores no esquema de Transporte.
 - **Alternativas:** Não armazenar o valor dos atributos no esquema de Transporte e buscar os valores nos atributos de origem sempre que houver a necessidade de uso.
 - **Vantagens:** Não é necessário buscar o valor em outras tabelas sempre que for utilizá-lo em aplicação, economizando na realização de JOINS e, consequentemente, gerando mais rapidez nas operações de buscas.
 - **Desvantagens:** Sempre que os atributos de origem dos atributos derivados forem atualizados, os atributos derivados também devem ser atualizados para manter a consistência. Como estes atributos, entretanto, não devem ser atualizados frequentemente, essa desvantagem é mínima.
 - **Justificativa 23 - Atributos derivados Custo Total e Tamanho Total de Lote de Entrega:** Os atributos derivados foram mapeados de modo a manter armazenados os valores no esquema de Lote de Entrega. Mesmo caso apontado na Justificativa 22.
 - **Alternativas:** Não armazenar o valor dos atributos na esquema de Lote de Entrega e buscar os valores nos atributos de origem sempre que houver a necessidade de uso.
 - **Vantagens:** Não é necessário buscar o valor em outras tabelas sempre que for

utilizá-lo em aplicação, economizando na realização de JOINS e, consequentemente, gerando mais rapidez nas operações de buscas.

- **Desvantagens:** Sempre que os atributos de origem dos atributos derivados forem atualizados, os atributos derivados também devem ser atualizados para manter a consistência. Como estes atributos, entretanto, não devem ser atualizados frequentemente, essa desvantagem é mínima.
- **Justificativa 24 - Atributo multivalorado Titular de Conta Bancária:** O mapeamento do atributo foi feito a partir da criação de uma tabela Titular, possuindo, como chave primária composta, os atributos Conta Bancária + Titular.
 - **Alternativas:** Armazenar os titulares em uma string na conta bancária, com cada valor separado por vírgula. Esta abordagem, entretanto, ocasionaria desperdício de memória dado o campo de tamanho imprevisível e dificultaria a adição, remoção e busca por titulares específicos.
 - **Vantagens:** Permite guardar mais de um valor para o mesmo atributo de uma entidade.
 - **Desvantagens:** É necessário realizar joins para fazer buscas.
- **Justificativa 25 - Mapeamento do relacionamento identificador “Realiza” entre Beneficiário e Solicitação de Aquisição - 1:N:** O mapeamento do relacionamento foi feito inserindo a chave primária de Beneficiário como parte da chave de Solicitação de Aquisição.
 - **Alternativas:** Uma vez que se trata de um relacionamento identificador 1:N, não devem existir outras alternativas válidas que garantam tanto a classificação da entidade fraca quanto a cardinalidade.
 - **Vantagens:** Mapeia todas as informações e características do relacionamento integralmente.
 - **Desvantagens:** Não há desvantagens.
- **Justificativa 26 - Mapeamento da chave de “Lote de Produto”:** A chave da entidade foi substituída por um identificador artificial chamado ID Lote, tornando a antiga chave, composta pelos atributos Produto, Produtor e Data/Hora Cadastro, um identificador secundário da entidade. Caso semelhante ao apontado na Justificativa 11.
 - **Alternativas:** Manter a chave composta anterior.
 - **Vantagens:** Torna-se não necessário a passagem de vários atributos como chave estrangeira. Assim como no caso da Conta Bancária (que recebeu um ID artificial), é válida a elaboração do ID Lote dado que existe um número considerável de referências ao Lote de Produto, poupando essas referências de carregar os três campos da antiga chave, assim: diminui o espaço de armazenamento ocu-

pado pelas chaves estrangeiras, simplifica as restrições de integridade referencial e torna os JOINS envolvendo Conta Bancária mais eficientes.

- **Desvantagens:** Um atributo a mais em Lote de Produto, perda do valor semântico da chave e perda da referência às entidades que formam a agregação (ex.: com a chave composta, era possível descobrir qual Produtor era responsável pelo Lote sendo referenciado por outra entidade diretamente pelo atributo presente nela).

5 Mudanças Relacionadas à Segunda Entrega

Após a segunda entrega, antes da implementação do banco de dados, foram realizados pequenos ajustes com base nas novas correções do monitor da disciplina e em outras ideias pertinentes do grupo para esta etapa.

- **Mudança dos campos CPF/CNPJ:** Para possibilitar um tratamento mais específico e adequado sobre o tipo de documento a ser registrado para as entidades Filantropo, Produtor Rural e Beneficiário, decidiu-se padronizar que o documento identificador de **Filantropo** e **Produtor Rural** é unicamente **CPF** e, de **Beneficiário**, é **CNPJ**.
- **Ajuste na Nota 9:** Mudança de "lotes" para "lote".
- **Adição da subseção 3.10:** Para justificar a omissão de um relacionamento apontada pelo monitor como possivelmente problemática, foi adicionada uma subseção que aponta a visão do grupo a respeito de como essa omissão é considerada a opção mais válida no contexto.
- **Adaptação da Justificativa 8:** Adaptação da escrita do trecho "atributo de critério como identificador da especialização", com a remoção do termo "identificador" devido ao sentido ambíguo gerado no contexto.
- **Adaptação da Justificativa 11:** Adição de uma explicação mais detalhada a respeito da abordagem adotada.
- **Adaptação da Justificativa 17:** Adaptação da escrita da vantagem da abordagem adotada de acordo com sugestão do monitor.
- **Adaptação da Justificativa 24:** Adição da análise sobre uma abordagem alternativa à adotada.

6 Implementação

Esta seção aborda a etapa de implementação do projeto, que engloba a criação do banco de dados, a inserção dos dados iniciais e o desenvolvimento do protótipo do sistema SOS Abobrinha. O objetivo é transformar o modelo relacional em código SQL para garantir a execução das regras de negócio elaboradas.

6.1 Ambiente de Desenvolvimento e Tecnologias

6.1.1 Front-End

Para o desenvolvimento da interface da aplicação, foram utilizadas as seguintes tecnologias:

- **HTML**: Responsável pela estruturação das páginas.
- **CSS**: Utilizado para a estilização e definição do layout da interface.
- **JavaScript**: Utilizado na implementação das funcionalidades e interações do lado do cliente.

6.1.2 Back-End

No desenvolvimento do back-end, foram utilizadas as seguintes tecnologias:

- **Python (Flask)**: Utilizado para a definição das rotas dos endpoints, inicialização do banco de dados e implementação das funções responsáveis pela manipulação e persistência dos dados.
- **PostgreSQL**: Sistema gerenciador de banco de dados relacional utilizado para o armazenamento das informações da aplicação.

6.1.3 Demais Tecnologias

Além das tecnologias citadas para o desenvolvimento da aplicação e de sua interface, é válido apontar que também foram usadas as seguintes tecnologias:

- **GitHub**: Usado como ferramenta de versionamento de código e de colaboração do grupo. O repositório do projeto está disponível em:
<https://github.com/danieljmanzano/bases-de-dados>
- **Docker (opcional)**: Ferramenta que facilita rodar o projeto em qualquer computador, sem exigir configurações manuais complexas. Usada por alguns integrantes em pontos específicos do desenvolvimento.

6.1.4 Requisitos para execução

Para executar o sistema, é necessário que os seguintes softwares estejam instalados no ambiente:

- Python 3.10 ou superior.
- PostgreSQL.
- Docker (opcional).

6.2 Organização de diretórios

```
1 trabalho3          # Diretório principal do projeto
2 |-- backend        # Arquivos da API e regras de negócio
```

```

3 | |-- app.py           # Arquivo principal de execução do servidor
4 | |-- config.py       # Configurações do sistema
5 | |-- db.py           # Conexão com o banco de dados
6 | |-- setup.db.py
7 |-- database         # Arquivos de banco de dados
8 | |-- consultas.sql   # Consultas exigidas no projeto
9 | |-- dados.sql       # Script de inserção inicial (povoamento)
10 | |-- esquema.sql     # Criação das tabelas e restrições
11 |-- docs             # Relatório da terceira etapa do projeto
12 |-- frontend         # Arquivos da interface de usuário
13 | |-- css
14 | | |-- styles.css   # Estilização das páginas
15 | |-- js             # Lógica da interface
16 | | |-- views        # Scripts específicos por página
17 | | | |-- cadastro
18 | | | | |-- passo1.js
19 | | | | |-- passo2.js
20 | | | | |-- passo3.js
21 | | | | |-- sucesso.js
22 | | | |-- consulta.js
23 | | | |-- menu.js
24 | | |-- app.js        # Script principal do frontend
25 | |-- index.html      # Página inicial e estrutura base
26 |-- README.md         # Guia com instruções sobre execução do projeto
27 |-- requirements.txt  # Dependências do projeto

```

6.3 Consultas

6.3.1 Consulta de quantidade adquirida e cadastrada por produto

Para cada produto, consultar a quantidade total adquirida (requisitada por beneficiários) e a quantidade total coletada (cadastrada em lotes) em um determinado mês.

- A consulta abaixo, localizada no arquivo `consultas.sql`, é responsável por retornar, para cada produto cadastrado no sistema, a quantidade total requisitada por beneficiários e a quantidade total cadastrada nos lotes dentro de um intervalo de tempo (no exemplo, o mês de junho de 2024).

Inicia-se com uma subconsulta não correlacionada que agrupa as requisições por lote, somando a `porcao_lote` de cada uma e filtrando apenas aquelas cuja `data_hora_aquisicao` pertence ao mês desejado. Essa subconsulta é então unida à tabela `Lote_de_Produto` por meio de um `LEFT JOIN`, de modo que lotes sem nenhuma requisição no período

também sejam considerados, com valor nulo na quantidade requisitada. A tabela `Produto` é, em seguida, unida à tabela `Lote_de_Produto`, também via `LEFT JOIN`, restringindo o cadastro ao mesmo intervalo de tempo, garantindo que produtos sem nenhum lote cadastrado no período apareçam na resposta com quantidade igual a zero. Por fim, agrupa-se pelo nome do produto e somam-se as quantidades requisitadas e cadastradas, ordenando o resultado alfabeticamente pelo nome do produto. O uso de intervalos (`>=` e `<`) para o filtro de mês, em vez de funções como `EXTRACT` ou `DATE_TRUNC`, permite que o otimizador utilize índices sobre as colunas de data, tornando a consulta mais eficiente.

```
1  SELECT
2      p.nome AS produto,
3      SUM(req_por_lote.quantidade_requisitada) AS
4      ↪ quantidade_total_adquirida,
5      SUM(lp.quantidade) AS quantidade_total_cadastrada
6  FROM Produto p
7  LEFT JOIN Lote_de_Produto lp
8      ON lp.produto = p.nome
9      AND lp.data_hora_cadastro >= '2024-06-01'
10     AND lp.data_hora_cadastro < '2024-07-01'
11  LEFT JOIN (
12      SELECT
13          r.lote,
14          SUM(r.porcao_lote) AS quantidade_requisitada
15      FROM Requisita r
16      WHERE r.data_hora_aquisicao >= '2024-06-01'
17            AND r.data_hora_aquisicao < '2024-07-01'
18      GROUP BY r.lote
19  ) req_por_lote
20  ON req_por_lote.lote = lp.id_lote
21  GROUP BY p.nome
22  ORDER BY p.nome;
```

6.3.2 Consulta de beneficiários de um lote específico

Consultar os beneficiários que adquiriram um determinado lote, considerando apenas requisições vinculadas a solicitações de aquisição aprovadas.

- A consulta abaixo, localizada no arquivo `consultas.sql`, é responsável por retornar o CNPJ, o nome e a classificação dos beneficiários que requisitaram um lote específico (no exemplo, o lote `LOTE-0001`), considerando somente as requisições associadas a

solicitações de aquisição com validação **APROVADA**.

Inicia-se unindo a tabela **Lote_de_Produto** à tabela **Requisita**, restringindo o resultado ao lote de interesse por meio da condição sobre **id_lote**. Em seguida, é feita uma junção com a tabela **Solicitacao_de_Aquisicao**, utilizando a chave composta por **data_hora** e **beneficiario**, de modo a recuperar a solicitação correspondente a cada requisição, filtrando apenas aquelas cuja validação seja **APROVADA**. Por fim, uma última junção com a tabela **Beneficiario** é realizada para obter os dados completos do beneficiário associado à solicitação aprovada. O uso de **UPPER** na comparação da validação garante que a consulta não seja sensível à caixa em que o valor foi armazenado.

```
1  SELECT DISTINCT
2      b.cnpj,
3      b.nome,
4      b.classificacao
5  FROM Lote_de_Produto lp
6  JOIN Requisita r
7      ON r.lote = lp.id_lote
8      AND lp.id_lote = 'LOTE-0001'
9  JOIN Solicitacao_de_Aquisicao sa
10     ON sa.data_hora = r.data_hora_aquisicao
11     AND UPPER(sa.validacao) = 'APROVADA'
12     AND sa.beneficiario = r.beneficiario
13  JOIN Beneficiario b
14     ON b.cnpj = sa.beneficiario;
```

6.3.3 Consulta de beneficiários que requisitaram todos os lotes de uma classificação (Divisão Relacional)

Consultar os beneficiários que já requisitaram pelo menos uma porção de todos os lotes de uma determinada classificação (no exemplo, **CONSUMO HUMANO**).

- A consulta abaixo, localizada no arquivo **consultas.sql**, implementa uma divisão relacional, retornando os beneficiários para os quais não existe nenhum lote da classificação alvo que ainda não tenha sido requisitado por eles.

A lógica é construída por meio de um duplo **NOT EXISTS**, técnica clássica para expressar divisão relacional em SQL. A subconsulta mais externa seleciona todo lote **lp** cuja classificação seja **CONSUMO HUMANO**; a subconsulta mais interna, por sua vez, verifica, para cada um desses lotes, se existe alguma requisição feita pelo beneficiário **b** associada a esse lote, unindo **Requisita** com **Solicitacao_de_Aquisicao** através da chave composta por **data_hora** e **beneficiario**. Dessa forma, o **NOT EXISTS** interno identifica lotes que o beneficiário nunca requisitou, enquanto o **NOT EXISTS**

externo garante que, para o beneficiário ser incluído no resultado, não pode existir nenhum lote daquela classificação que ele não tenha requisitado — ou seja, ele requisitou todos.

Em termos de eficiência, a comparação da classificação é feita de forma direta (`lp.classificacao = 'CONSUMO HUMANO'`), sem o uso de `UPPER()`, já que essa coluna é sempre armazenada em letras maiúsculas (conforme padronizado em `dados.sql` e no mapeamento da camada de aplicação em `app.py`). Evitar funções sobre a coluna filtrada permite que o índice `idx_lote_produto_classificacao` seja efetivamente utilizado pelo otimizador, já que a aplicação de `UPPER()` sobre a coluna impediria o uso desse índice, forçando uma varredura sequencial.

```
1      SELECT
2          b.cnpj,
3          b.nome
4      FROM Beneficiario b
5      WHERE NOT EXISTS (
6          SELECT 1
7          FROM Lote_de_Produto lp
8          WHERE lp.classificacao = 'CONSUMO HUMANO'
9              AND NOT EXISTS (
10                 SELECT 1
11                 FROM Requisita r
12                 JOIN Solicitacao_de_Aquisicao sa
13                     ON sa.data_hora      = r.data_hora_aquisicao
14                     AND sa.beneficiario = r.beneficiario
15                 WHERE r.lote              = lp.id_lote
16                     AND sa.beneficiario = b.cnpj
17             )
18      );
```

6.3.4 Consulta de lotes com preço por unidade acima da média do produtor

Consultar os lotes cujo preço por unidade é maior que a média do preço por unidade dos demais lotes do mesmo produtor.

- A consulta abaixo, localizada no arquivo `consultas.sql`, é responsável por identificar, dentre os lotes cadastrados, aqueles cujo preço por unidade — calculado pela razão entre `custo_producao` e `quantidade` — é superior à média do preço por unidade praticado pelo mesmo produtor.

Para isso, utilizou-se uma subconsulta correlacionada: para cada lote da consulta externa, calcula-se, na subconsulta interna, a média do preço por unidade de todos

os lotes pertencentes ao mesmo produtor, por meio da condição `lp2.produtor = lp.produtor`, que faz referência à linha da consulta externa. O lote é incluído no resultado apenas se seu preço por unidade individual for maior que essa média. O resultado final é ordenado por produtor e, dentro de cada produtor, de forma decrescente pelo preço por unidade. Vale observar que a expressão `lp.custo_producao / lp.quantidade` pressupõe que o campo quantidade seja sempre positivo, o que é garantido pela restrição CHECK (`quantidade > 0`) definida no `esquema.sql`. Dessa forma, não há risco de divisão por zero durante a execução da consulta.

Em termos de eficiência, como a subconsulta é reexecutada para cada linha da consulta externa, é importante que exista um índice sobre a coluna `produtor` da tabela `Lote_de_Produto` (`idx_lote_produto_produtor`), garantindo que a busca pelos lotes de cada produtor seja feita por meio de um *Index Scan*, em vez de um *Seq Scan*. O custo dessa abordagem tende a ser alto em cenários com poucos produtores que possuam muitos lotes cada, porém é aceitável no cenário esperado pelo sistema, em que há muitos produtores com poucos lotes cada. Caso o volume de lotes por produtor cresça significativamente, uma alternativa seria o uso de uma função de janela (*window function*), que permitiria calcular a média uma única vez por grupo, em vez de recalculá-la para cada linha.

```
1  SELECT
2      lp.id_lote,
3      lp.produto,
4      lp.produtor,
5      lp.custo_producao,
6      lp.quantidade,
7      lp.custo_producao / lp.quantidade AS preco_por_unidade
8  FROM Lote_de_Produto lp
9  WHERE (lp.custo_producao / lp.quantidade) > (
10      SELECT AVG(lp2.custo_producao / lp2.quantidade)
11      FROM Lote_de_Produto lp2
12      WHERE lp2.produtor = lp.produtor
13  )
14  ORDER BY lp.produtor, preco_por_unidade DESC;
```

6.3.5 Consulta de beneficiários com requisição acima da média

Consultar os beneficiários cujo volume total requisitado (em m^3) é maior que o volume médio requisitado, considerando os beneficiários que já possuem ao menos uma requisição registrada.

- A consulta abaixo, localizada no arquivo `consultas.sql`, é responsável por identi-

ficar os beneficiários cujo volume total das porções requisitadas (tabela **Requisita**) é maior que a média geral de volume por beneficiário.

Para isso, a consulta externa une as tabelas **Beneficiario** e **Requisita**, agrupando o resultado por beneficiário e somando o volume de **porcao_lote** de cada um. Essa soma é então comparada, na cláusula **HAVING**, com o valor retornado por uma subconsulta não correlacionada, isto é, uma subconsulta que não faz referência a nenhuma coluna da consulta externa, sendo executada apenas uma vez. Essa subconsulta, por sua vez, calcula o volume total requisitado por cada beneficiário em uma subconsulta interna e, em seguida, calcula a média desses valores com **AVG**, obtendo assim a média geral de volume requisitado entre todos os beneficiários.

Diferentemente de filtros como "beneficiários que nunca requisitaram", que podem ser reescritos como uma junção externa (*anti-join*) sem perda de expressividade, essa consulta depende intrinsecamente do uso de subconsulta, já que o valor de comparação (a média geral do volume) precisa ser calculado de forma agregada e independente antes de ser comparado ao agregado de cada beneficiário individualmente — algo que não pode ser obtido apenas com junções e agrupamento simples.

```
1  SELECT
2      b.cnpj,
3      b.nome,
4      SUM(r.porcao_lote) AS quantidade_total_requisitada
5  FROM Beneficiario b
6  JOIN Requisita r
7      ON r.beneficiario = b.cnpj
8  GROUP BY b.cnpj, b.nome
9  HAVING SUM(r.porcao_lote) > (
10     SELECT AVG(quantidade_por_beneficiario)
11     FROM (
12         SELECT SUM(r2.porcao_lote) AS quantidade_por_beneficiario
13         FROM Requisita r2
14         GROUP BY r2.beneficiario
15     ) AS medias_por_beneficiario
16 );
```

6.4 Aplicação

A aplicação desenvolvida para o projeto SOS Abobrinha implementa a funcionalidade de inserção de lotes, destinada aos produtores, e a visualização de lotes disponíveis, destinada aos beneficiários.

6.4.1 Funcionalidade de inserção de lotes

O fluxo de inserção do lote pode ser resumido em três principais etapas: listagem dos produtos, verificação do produtor e criação do lote.

Na primeira etapa, realiza-se uma busca na tabela Produto para identificar todos produtos cadastráveis, retornando-os para exibição e seleção na interface.

Na segunda etapa, a partir do CPF inserido, é executada uma consulta na tabela Produtor_Rural para validar se o produtor está devidamente cadastrado no sistema.

Por fim, o usuário preenche as informações restantes do lote e finaliza o cadastro no banco de dados do sistema.

- A função `listar_produtos()`, no arquivo `app.py`, é responsável por listar todos os produtos cadastrados. A query `SELECT` retorna uma lista de tuplas da tabela Produto, contendo o nome e o tipo do produto, ordenada de maneira alfabética.

```
1 @app.get("/api/produtos")
2 def listar_produtos():
3     with get_cursor() as cursor:
4         cursor.execute("SELECT nome, tipo FROM Produto ORDER BY
5             ↪ nome")
6         produtos = cursor.fetchall()
7     return jsonify(produtos)
```

- A função `buscar_produto()` realiza a busca do produtor. Para esse objetivo, a função faz uma validação inicial do CPF fornecido e executa o comando `SELECT`. A query retorna uma tupla contendo o CPF e o nome do produtor correspondente ao parâmetro (CPF).

```
1 @app.get("/api/produtores")
2 def buscar_produto():
3     cpf = somente_digitos(request.args.get("cpf", ""))
4     if not cpf:
5         return jsonify({"erro": "Informe o CPF do produtor."}),
6             ↪ 400
7
8     with get_cursor() as cursor:
9         cursor.execute("SELECT cpf, nome FROM Produtor_Rural
10             ↪ WHERE cpf = %s", (cpf,))
11         produtor = cursor.fetchone()
12
13     if not produtor:
14         return jsonify({"erro": "Produtor não encontrado."}), 404
```

```

13
14         return jsonify(produtor)

```

- A função `criar_lote()` é responsável por inserir tuplas na tabela Lote de Produto. A função realiza uma primeira verificação das entradas recebidas, como os campos obrigatórios e formatação da data. Após isso, a query `INSERT` adiciona uma nova tupla. A chave primária é gerada de maneira automática no formato `LOTE-XXXX`. Por uma decisão de escopo da interface, o campo `classificacao` é definido automaticamente como `'CONSUMO HUMANO'` no momento do cadastro, de forma que o lote já apareça imediatamente no filtro de visualização destinado aos beneficiários de consumo humano, sem depender da etapa de triagem para fins de demonstração do protótipo. Em caso de falha de integridade referencial, o código identifica qual chave falhou e retorna o erro adequado..

```

1     @app.post("/api/lotos")
2     def criar_lote():
3         dados = request.get_json(silent=True) or {}
4
5         campos_obrigatorios = ["produto", "cpf", "quantidade",
6         ↪ "data_colheita", "validade", "localizacao"]
7         faltantes = [campo for campo in campos_obrigatorios if not
8         ↪ dados.get(campo)]
9         if faltantes:
10             return jsonify({"erro": f"Campos obrigatórios faltando:
11             ↪ {' ', ' '.join(faltantes)}"}), 400
12
13         try:
14             data_colheita = data_br_para_iso(dados["data_colheita"])
15             validade = data_br_para_iso(dados["validade"])
16         except ValueError:
17             return jsonify({"erro": "Datas devem estar no formato
18             ↪ dd/mm/aaaa."}), 400
19
20         cpf = somente_digitos(dados["cpf"])
21
22         try:
23             with get_cursor(commit=True) as cursor:
24                 cursor.execute(
25                     """
26                     INSERT INTO Lote_de_Produto

```

```

23         (id_lote, produto, produtor, quantidade,
24         ↪ data_colheita, validade, localizacao,
25         ↪ classificacao)
26     VALUES
27     ('LOTE-' ||
28     ↪ LPAD(nextval('seq_lote_id')::text, 4,
29     ↪ '0'),
30     %s, %s, %s, %s, %s, %s, %s)
31     RETURNING id_lote
32     """
33     (
34     ↪ dados["produto"],
35     ↪ cpf,
36     ↪ dados["quantidade"],
37     ↪ data_colheita,
38     ↪ validade,
39     ↪ dados["localizacao"],
40     ↪ "CONSUMO HUMANO",
41     ↪ ),
42     ↪ )
43     novo_lote = cursor.fetchone()
44     except errors.ForeignKeyViolation as erro:
45     ↪ constraint = erro.diag.constraint_name
46     ↪ if constraint == "fk_lote_produto_produto":
47     ↪     ↪ return jsonify({"erro": "Produto não encontrado."}),
48     ↪     ↪ 404
49     ↪ if constraint == "fk_lote_produto_produto":
50     ↪     ↪ return jsonify({"erro": "Produtor não encontrado."}),
51     ↪     ↪ 404
52     ↪ return jsonify({"erro": "Referência inválida."}), 404
53     ↪ return jsonify({"id_lote": novo_lote["id_lote"]}), 201

```

6.4.2 Funcionalidade de visualização de produtos

- A função `listar_lotes()` busca os lotes cadastrados. A query `SELECT` realiza uma operação de `JOIN` para juntar as tabelas de `Lote_de_Produto` e `Produtor`, filtrando, a partir da cláusula `WHERE`, a classificação de cada lote de produto. O retorno é uma lista de tuplas contendo o identificador, produto, quantidade, validade, localização e nome do produtor, ordenada pela data de cadastro.

```
1  @app.get("/api/lotes")
2  def listar_lotes():
3      classificacao = request.args.get("classificacao", "")
4      classificacao_bd = CLASSIFICACOES_VALIDAS.get(classificacao)
5      if not classificacao_bd:
6          return jsonify({"erro": "Classificação inválida."}), 400
7
8      with get_cursor() as cursor:
9          cursor.execute(
10              """
11              SELECT lp.id_lote AS id, lp.produto,
12              ↪ lp.classificacao, lp.quantidade,
13              ↪ lp.validade, lp.localizacao, pr.nome AS
14              ↪ produtor
15              FROM Lote_de_Produto lp
16              JOIN Produtor_Rural pr ON pr.cpf = lp.produtor
17              WHERE lp.classificacao = %s
18              ORDER BY lp.data_hora_cadastro DESC
19              """,
20              (classificacao_bd,),
21          )
22      lotes = cursor.fetchall()
23
24      for lote in lotes:
25          lote["classificacao"] =
26          ↪ CLASSIFICACOES_EXIBICAO.get(lote["classificacao"],
27          ↪ lote["classificacao"])
28          if lote["validade"]:
29              lote["validade"] =
30              ↪ lote["validade"].strftime("%d/%m/%Y")
31
32      return jsonify(lotes)
```

6.5 Tratamento de Vulnerabilidades

A aplicação realiza a prevenção de ataques de **SQL Injection** durante as operações de acesso ao banco de dados por meio do uso de consultas parametrizadas da biblioteca **psycopg2**, como é fortemente recomendado em sua documentação. Como a comunicação com o SGBD PostgreSQL ocorre por meio de comandos SQL enviados pela aplicação, é fundamental garantir que dados fornecidos pelos usuários não sejam interpretados como

parte dessas instruções. Para isso, em vez de concatenar diretamente os valores recebidos às strings SQL, as consultas utilizam marcadores de posição (`%s`) e os dados são passados separadamente no segundo parâmetro do método `execute()`. Dessa forma, o driver envia ao PostgreSQL o comando SQL e os parâmetros de maneira independente, fazendo com que os valores informados pelo usuário sejam tratados apenas como dados e não como parte da instrução SQL. Isso impede que entradas maliciosas alterem a estrutura da consulta ou executem comandos não autorizados no banco de dados. Além disso, algumas entradas são previamente validadas e sanitizadas, como a remoção de caracteres não numéricos de CPFs, a restrição de valores válidos para classificações e a formatação correta de datas, reforçando a segurança da interação entre a aplicação e o SGBD.

Além disso, a aplicação também adota medidas para prevenir ataques de **Cross-Site Scripting (XSS)** na camada de apresentação dos dados retornados pelo sistema. Essa vulnerabilidade ocorre quando dados armazenados ou recuperados do banco de dados são inseridos na página como HTML, permitindo que códigos JavaScript maliciosos sejam executados no navegador de outros usuários. Para evitar esse problema, os dados recebidos da API não são inseridos diretamente em propriedades como `innerHTML`. Diferente disso, o código cria a estrutura HTML estaticamente e preenche os conteúdos dinâmicos utilizando a propriedade `textContent`, que trata os valores apenas como texto. Dessa forma, mesmo que um usuário tente cadastrar conteúdo contendo tags HTML ou scripts e esse conteúdo seja posteriormente recuperado do banco de dados, tais elementos serão exibidos literalmente na página, sem serem interpretados ou executados pelo navegador. Isso garante que os dados armazenados e consultados no SGBD possam ser exibidos com segurança na interface da aplicação.

6.6 Controle Transacional

Para analisar como o sistema gere as transações do banco de dados, é necessário avaliar dois momentos distintos: a inicialização do banco de dados e a execução dos endpoints.

No momento em que o banco de dados é inicializado, cria-se a estrutura das tabelas através do arquivo `esquema.sql` e, logo a seguir, realiza-se a inserção dos dados iniciais com o `dados.sql`. Todo este bloco de comandos (as operações `CREATE` e `INSERT`) é executado dentro de uma única transação. Esta abordagem garante que, caso ocorra alguma falha no meio do processo, seja na criação de uma tabela ou na inserção de um registo, o banco de dados desfça todas as alterações imediatamente (rollback). O sistema retorna, assim, ao estado inicial, o que evita tabelas criadas pela metade ou dados corrompidos, impedindo que o banco assuma um estado inconsistente ou parcialmente configurado.

Nas rotas da API, cada endpoint executa apenas uma única operação no banco

(seja uma leitura ou uma alteração). Por este motivo, a principal preocupação é a concorrência, ou seja, o acesso simultâneo de múltiplos usuários ao sistema. Como a aplicação atende produtores e beneficiários simultaneamente, a ausência de um isolamento correto entre as transações permitiria que os dados fossem lidos de forma inconsistente. Como cada endpoint executa apenas um comando isolado de `SELECT` ou `INSERT`, a única anomalia concorrente capaz de causar problemas é a chamada leitura suja (*dirty read*). Esse problema ocorre se um processo acessa um dado temporário de outra operação que falha antes de ser finalizada e acaba sendo revertida.

Para mitigar este risco, a aplicação utiliza o nível de isolamento `Read Committed`. Esse modelo, que é o padrão do PostgreSQL e da biblioteca `psycopg2`, garante que apenas os dados gravados e confirmados de maneira definitiva fiquem disponíveis para leitura, evitando inconsistências.

7 Conclusão

O desenvolvimento do projeto SOS Abobrinha permitiu aplicar, de forma integrada, os principais conceitos da disciplina, desde o levantamento de requisitos e a modelagem conceitual até a implementação do banco de dados e do protótipo funcional. A experiência de revisar e corrigir o modelo a cada entrega, incorporando o feedback do monitor, reforçou a importância de uma modelagem cuidadosa nas etapas iniciais do projeto.

Entre os pontos de maior dificuldade, destacam-se a modelagem das entidades com generalização, a eliminação de ciclos redundantes no MER e a justificativa das decisões de mapeamento. Na implementação, a integração entre Flask e PostgreSQL com controle transacional e prevenção de SQL Injection também demandou atenção às boas práticas de segurança.

Como sugestão para turmas futuras, recomenda-se incentivar a implementação de ao menos uma funcionalidade por perfil de usuário do sistema, tornando o protótipo mais representativo da proposta original e a apresentação final mais dinâmica.

8 Referências

References

- [1] JORNAL DA USP. **Perdas pós-produção e pré-consumo geram um grande desperdício de alimentos no Brasil**. Jornal da USP, São Paulo. Disponível em: <https://jornal.usp.br/atualidades/perdas-pos-producao-e-pre-consumo-geram-um-grande-desperdicio-de-alimentos-no-> Acesso em: 11 mar. 2026.

- [2] PSYCOPG. **Passing parameters to SQL queries**. Disponível em: <https://www.psycopg.org/docs/usage.html#query-parameters>. Acesso em: 20 jun. 2026.